M.TECH PROJECT REPORT

# Can Self-Attention Models Capture Context Effects?

Higher-order context effects, a comparison with DeepHalo, and a first look inside the model

Ajaz Hussain
DA25M006

M.Tech, Data Science and Artificial Intelligence
Indian Institute of Technology Madras

Guide: Prof. Suryanarayana Sankagiri

**Abstract.** When we choose something, we hardly ever look at it alone: whether we like a hotel or a phone depends on what else is shown next to it. In choice models every option gets a score called its *utility*, which says how attractive it is, and a *context effect* means that this utility changes with the other options in the set. Models like CDM and LCL capture only the first-order part of context, where each other item (or the slate average) pushes an item's utility up or down. In this project we check whether a self-attention model learns context effects of a known order from choices alone, and whether it does so better than DeepHalo (NeurIPS 2025), a neural model built to control the order of interaction. First, on first-order LCL data, a 1-layer self-attention model recovers the planted context matrix $A$ with cosine 0.990 and 93% of its true strength, as well as a mean-pool network and much better than a flat MLP. Second, on synthetic data with planted effects of order 1, 2 and 3, a tuned 3-layer self-attention model beats a tuned DeepHalo model on every one of 30 seeds at orders 2 and 3, capturing 16.8% to 32.3% of the part of the effect that the slate's mean and standard deviation cannot explain, against 5.7% to 9.8% for DeepHalo; this holds for two different kernels, while an earlier advantage on similarity effects most likely came from the kernel used to generate the data. Third, the remaining gap to the best possible model is mainly a data limit: with 1,000,000 slates self-attention captures 38.2% to 53.1% of this hard part, while below about 100,000 slates no model beats the mean-and-sd summary. Fourth, a look inside the trained models shows that on LCL data attention simply averages the slate, while on higher-order data its heads attend to the items close in the features that define the effect. On real data, we find no usable context in London travel-mode choice (LPMC) and a small, shallow context gain in Expedia hotel bookings, where tuned self-attention and DeepHalo tie.

# Contents

**1 Introduction and Related Work** **1**
1.1 Context Effects . . . 1
1.2 Basic Terms . . . 1
1.3 Context Effects in Choice and IIA . . . 2
1.4 The Order of a Context Effect . . . 2
1.5 Related Work . . . 2
1.6 Research Questions . . . 3

**2 Models and Setup** **3**
2.1 First-Order Context Models . . . 3
2.2 Our Self-Attention Model . . . 3
2.3 DeepHalo . . . 4
2.4 Synthetic Data . . . 5
2.5 Metrics and a Fair Comparison . . . 5

**3 Experiments and Results** **6**
3.1 First-Order Context: Recovering the LCL Matrix . . . 6
3.2 Higher-Order Context: Self-Attention against DeepHalo . . . 7
3.3 Training-Set Size . . . 8
3.4 A Look Inside the Models . . . 8
3.5 Depth and the Order of Interaction . . . 9
3.6 Real Data: LPMC and Expedia . . . 10
3.7 Lessons Learned Along the Way . . . 11

**4 Conclusion, Limitations and Next Steps** **11**

**Bibliography** **12**

# 1 Introduction and Related Work

## 1.1 Context Effects

**Context effect**

A context effect is when our liking for an option changes because of the *other* options shown along with it, even though the option itself has not changed. In choice models, liking is measured by the option's utility, so a context effect means that the utility of an item depends on the whole choice set.

Let us start with a simple situation. A café sells coffee in two sizes, small at Rs 100 and large at Rs 200, and most people pick the small one. Now the café adds a medium cup at Rs 190. Almost nobody buys the medium, but many people now shift to the large, because for just Rs 10 more they get much more coffee. Only the cups around the large one have changed, and that was enough to change what people chose.

This idea is much older than choice modelling. Tversky [1977] argued that even our sense of *similarity* depends on context. He described each object by a set of features, and the similarity of $a$ to $b$ as a contrast between shared and distinct features,

$$s(a, b) = \theta f(A \cap B) - \alpha f(A \setminus B) - \beta f(B \setminus A), \tag{1.1}$$

where the weight of a feature is not fixed: by his *diagnosticity principle*, a feature becomes more important when it helps divide the objects in front of us into groups. When people were asked which country is most similar to Austria, 49% chose Sweden from Sweden, Poland and Hungary, but only 14% when Poland was replaced by Norway. Goldstone et al. [1997] showed that this happens even when people compare just two things: which features matter changes with the other objects on the screen. We should also be careful: when Evers and Lakens [2014] repeated Tversky's study, they found only a small effect, which disappeared once they ruled out a simpler explanation. So context effects are real, but they can be small, and something that looks like a context effect may have a different cause. This lesson matters for our project as well.

## 1.2 Basic Terms

Table 1.1 defines the terms used throughout this report.

**Table 1.1.** Basic terms.

| Term | Meaning |
|---|---|
| Item, slate | An option, and the set of options shown together (the choice set $S$). |
| Utility | The score $u_i(S)$ of item $i$: the higher it is, the more likely the item is chosen. |
| Softmax | The rule that turns utilities into choice probabilities: $P(i \mid S) = e^{u_i} / \sum_{j \in S} e^{u_j}$, so the probabilities add up to 1. |
| Order of an effect | How many *other* items act together on one item's utility (Section 1.4). |
| Oracle | The true model that generated the synthetic data; its loss is the best any model can reach. |
| Blindfolded twin | Our self-attention model with each item allowed to attend only to itself, so it sees no context. |
| Mean+sd pool | A model that sees each item plus only the mean and standard deviation of the others. |
| Hard part | The part of a context effect that the mean+sd pool cannot explain. |
| Seed | One random dataset and initialisation; results are averaged over seeds. |
| Cross-entropy, nats | How far a model's choice probabilities are from the true ones (lower is better), measured in nats, the unit of the natural logarithm. |
| Kernel | A function that turns a distance into a closeness score: 1 for very close items, near 0 for far ones. |

## 1.3 Context Effects in Choice and IIA

Three context effects in choice have been reported again and again: the *attraction (decoy) effect*, where adding a clearly worse option raises the share of the option it resembles [Huber et al., 1982]; the *compromise effect*, where an option between two extremes is chosen more often [Simonson, 1989]; and the *similarity effect*, where a new option takes its share mostly from similar options [Tversky, 1972]. All three break the rule of *independence of irrelevant alternatives* (IIA) of Luce [1959], which says that the ratio of the chances of choosing $i$ and $j$ should not depend on what else is offered:

$$\frac{P(i \mid S)}{P(j \mid S)} = \frac{P(i \mid S')}{P(j \mid S')} \qquad \text{for every } S' \supseteq \{i, j\}. \tag{1.2}$$

The multinomial logit, the most widely used choice model, follows IIA by construction, and so does any ranking model that scores each item on its own; when IIA does not hold, predictions for new choice sets can go badly wrong [Seshadri et al., 2019]. Context can also act on features: among expensive restaurants a diner may care about the wine list, among fast-food places about speed [Tomlinson and Benson, 2021].

## 1.4 The Order of a Context Effect

A useful way to organise context effects is to break the utility of item $j$ in slate $S$ into pieces by how many other items each piece depends on [Batsell and Polking, 1985; Seshadri et al., 2019; Zhang et al., 2025], and to turn utilities into choice probabilities with a softmax:

$$u_j(S) = v_j(\varnothing) + \sum_{k \in S \setminus j} v_j(\{k\}) + \sum_{\{k,l\} \subseteq S \setminus j} v_j(\{k,l\}) + \cdots, \qquad P(j \mid S) = \frac{e^{u_j(S)}}{\sum_{k \in S} e^{u_k(S)}}. \tag{1.3}$$

Here $v_j(\varnothing)$ is the item's own value, the first-order terms say how one other item changes $u_j$, the second-order terms how a pair acts together beyond what each does alone, and so on. The attraction effect is a positive first-order term, the similarity effect a negative first-order term, and the compromise effect a positive second-order term, since an option is a compromise only with respect to *two* extremes [Zhang et al., 2025]. In this report the order always counts only the *other* items, as in DeepHalo.

## 1.5 Related Work

**Choice models.** The Bradley–Terry model [Bradley and Terry, 1952] gives each item one score and compares pairs; the multinomial logit (MNL) [McFadden, 1974] extends it to slates and keeps only the zeroth-order term. The Context-Dependent Model (CDM) [Seshadri et al., 2019] adds first-order pairwise pushes between item identities, and the Linear Context Logit (LCL) [Tomlinson and Benson, 2021] lets the slate's average feature vector change the weights on features. Both are first order.

**Set-aware neural models.** Self-attention [Vaswani et al., 2017] lets every item of a list look at every other item, and is used to re-rank lists [Pobrotyn et al., 2020]. FATE and FETA [Pfannschmidt et al., 2022] aggregate the set and then score each item, TCNet [Wang et al., 2023] uses a one-layer Transformer for choice, and Ko and Li [2023] show that a single attention head is a low-rank generalisation of a first-order (halo) MNL. These works report prediction accuracy, but do not check *which* context effect was learned. One attention layer with a feed-forward network can approximate any permutation-equivariant function given enough width [Kajitsuka and Sato, 2024], so the question is not whether attention can represent context effects, but whether it learns them from a finite number of choices.

**DeepHalo.** DeepHalo [Zhang et al., 2025] is the closest work: a neural choice model whose layers are built so that the order of interaction it can represent is controlled by depth, up to $2^{L-1}$ with $L$ quadratic layers. It is the strongest neural context model we know, so we use it as our baseline.

**Accuracy is not recovery.** High accuracy does not prove that context was learned. In our early experiments on Expedia hotel searches, a model that used only position bias reached an NDCG@10 of 0.3922, slightly above the 0.3848 of the full attention model. We therefore work mainly with synthetic data, where the true utilities are known. The code and notebooks of our earlier experiments are at github.com/ajazhsn/Ajaz_Hussain_MTP-.

### 1.6 Research Questions

1. **RQ1.** On first-order (LCL) data, does a self-attention model that predicts well also recover the true context effect?
2. **RQ2.** For effects of order 1, 2 and 3, how much does self-attention learn, and does it learn more than DeepHalo?
3. **RQ3.** What limits what is learned: the amount of data, or the depth and size of the model? And where inside the model is context computed?
4. **RQ4.** What do the same models and controls find on two real datasets, LPMC and Expedia?

## 2 Models and Setup

**Table 2.1.** Main symbols; others are defined where used.

| | | | |
|---|---|---|---|
| $S$, $n$ | slate and its size | $\mathbf{x}_i \in \mathbb{R}^p$ | feature vector of item $i$ |
| $u_i(S)$ | utility of item $i$ in slate $S$ | $P(i \mid S)$ | choice probability (softmax of $u$) |
| $\boldsymbol{\theta}$ | weights of the item's own part | $A$ | LCL context matrix |
| $\bar{\mathbf{x}}_S$ | mean feature vector of the slate | $k$ | order of the planted effect |
| $\lambda$, $s$ | strength and sign of the effect | $L$, $d$, $R$ | layers, width, heads (self-attention) |
| $a_{ij}^r$ | attention weight from $i$ to $j$ in head $r$ | $H$ | interaction heads (DeepHalo) |
| CE | test cross-entropy against the true $P$ | $\hat{A}$, $\hat{\boldsymbol{\theta}}$ | parameters read from a trained model |

### 2.1 First-Order Context Models

In the MNL, $u_i = \boldsymbol{\theta} \cdot \mathbf{x}_i$ and the ratio $P(i \mid S)/P(j \mid S) = e^{u_i - u_j}$ does not depend on $S$, which is exactly IIA. The CDM [Seshadri et al., 2019] adds a push from every other item, $u_i(S) = \sum_{k \in S \setminus i} \mathbf{t}_i \cdot \mathbf{c}_k$, with a "target" vector $\mathbf{t}_i$ and a "context" vector $\mathbf{c}_k$ per item. The LCL [Tomlinson and Benson, 2021] works with features:

$$u_i(S) = (\boldsymbol{\theta} + A\bar{\mathbf{x}}_S) \cdot \mathbf{x}_i = \boldsymbol{\theta} \cdot \mathbf{x}_i + \mathbf{x}_i^\top A\, \bar{\mathbf{x}}_S, \qquad \bar{\mathbf{x}}_S = \tfrac{1}{n} \textstyle\sum_{j \in S} \mathbf{x}_j. \tag{2.1}$$

The entry $A_{pq}$ says how the slate average of feature $q$ changes how much people value feature $p$; with $A = 0$ the LCL is the MNL. Since $\bar{\mathbf{x}}_S$ is an average, each term depends on one other item at a time, so both CDM and LCL are first order and cannot express an effect that needs two or three items together.

### 2.2 Our Self-Attention Model

The model is a Transformer encoder applied to the slate as a set (Figure 2.1, left). Each item is embedded as $\mathbf{h}_i^{(0)} = \mathrm{LN}(W_e \mathbf{x}_i + \mathbf{b}_e)$, with no positional encoding, so shuffling the items does not change their scores. In layer $\ell$, each of $R$ heads forms queries, keys and values and mixes in the other items:

$$a_{ij}^r = \frac{\exp(\mathbf{q}_i^r \cdot \mathbf{k}_j^r / \sqrt{d_r})}{\sum_{m \in S} \exp(\mathbf{q}_i^r \cdot \mathbf{k}_m^r / \sqrt{d_r})}, \qquad \mathrm{MHA}_i = W_O[\textstyle\sum_j a_{ij}^1 \mathbf{v}_j^1; \ldots; \sum_j a_{ij}^R \mathbf{v}_j^R], \tag{2.2}$$

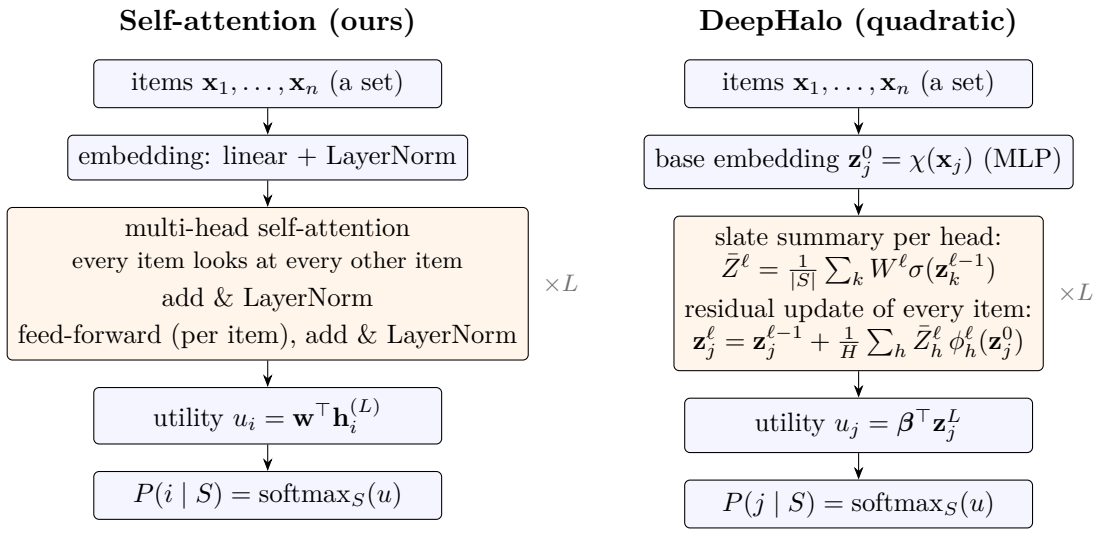



**Figure 2.1.** The two architectures. In self-attention each item forms its own weights over the other items; in DeepHalo every item sees the same $H$ averaged numbers per layer and differs only in how its own base embedding responds to them.

where $\mathbf{q}_i^r = W_Q^r \mathbf{h}_i^{(\ell-1)}$, and similarly for keys and values. Attention and a two-layer ReLU feed-forward network (hidden size $2d$) are each wrapped in a residual connection followed by layer normalisation:

$$\tilde{\mathbf{h}}_i = \mathrm{LN}\big(\mathbf{h}_i^{(\ell-1)} + \mathrm{MHA}_i\big), \qquad \mathbf{h}_i^{(\ell)} = \mathrm{LN}\big(\tilde{\mathbf{h}}_i + W_2 \,\mathrm{ReLU}(W_1 \tilde{\mathbf{h}}_i + \mathbf{b}_1) + \mathbf{b}_2\big). \tag{2.3}$$

After $L$ layers, $u_i(S) = \mathbf{w}^\top \mathbf{h}_i^{(L)} + b$, and the model is trained with the negative log-likelihood of the observed choices. In simple words, each item asks "which other items are relevant to me?" and takes a weighted average of what they carry; the residual connection adds this new information to what the item already had, and layer normalisation keeps the numbers in a stable range. Attention is the only step that mixes information across items. The tuned model has $L = 3$ layers, width $d = 32$, $R = 2$ heads (head size 16), feed-forward size 64 and 25,953 parameters. These settings were not our first choice: our first models had one or two layers, and the final values came from tuning 22 configurations (Section 2.5). The *blindfolded twin* uses the same blocks (one layer, width 32, two heads) with the attention masked so that $a_{ij}^r = 0$ for $j \neq i$; its utility depends only on the item's own features, so any gain of the full model over the twin must come from context.

## 2.3 DeepHalo

DeepHalo [Zhang et al., 2025] starts from the order-wise decomposition of Section 1.4, written for feature-based utilities, and builds it layer by layer (Figure 2.1, right). Each item is embedded by a shared network, $\mathbf{z}_j^0 = \chi(\mathbf{x}_j)$ (in the released code a three-layer ReLU network with layer normalisation). In layer $\ell$ the current representations are projected into $H$ *interaction heads* and averaged, and each item adds, for every head, a nonlinear function of its own base embedding scaled by that head's summary:

$$\bar{Z}^\ell = \frac{1}{|S|} \sum_{k \in S} W^\ell \, \sigma(\mathbf{z}_k^{\ell-1}) \in \mathbb{R}^H, \qquad \mathbf{z}_j^\ell = \mathbf{z}_j^{\ell-1} + \frac{1}{H} \sum_{h=1}^{H} \bar{Z}_h^\ell \, \phi_h^\ell(\mathbf{z}_j^0), \qquad u_j = \boldsymbol{\beta}^\top \mathbf{z}_j^L. \tag{2.4}$$

In simple words, DeepHalo first makes a short summary of the whole slate ($H$ numbers per layer), and then lets each item react to that summary in its own way; stacking layers builds interactions of higher and higher order. With $\sigma$ the identity (the paper's Eq. 4, the *linear* form) each layer raises the attainable order by one, so $L$ layers reach order $L$. With a quadratic $\sigma$ (the paper's Eq. 6) the order doubles at every layer and reaches $2^{L-1}$. The released code implements the linear

form; it defines two linear maps for the quadratic activation but never applies them. Our first experiments used the code unchanged; we then wired those maps in as $\sigma(\mathbf{z}) = (A'\mathbf{z}+\mathbf{a})\odot(B'\mathbf{z}+\mathbf{b})$, a learnable quadratic that contains the square used in the paper, and call this version **DHQ**. All DeepHalo results below use DHQ. Its settings were also chosen by tuning: we first ran the paper's own configuration ($L = 4$, $d = 32$, $H = 8$) and smaller variants, and the tuning selected $L = 4$, $d = 16$, $H = 8$ (11,713 parameters).

## 2.4 Synthetic Data

**Generator.** Every slate has $n = 10$ items, each with $p = 6$ features drawn from a standard normal distribution. For each seed a weight vector $\boldsymbol{\theta}$ is drawn and scaled to length 1. The true utility is the item's own part plus a planted context part, and the chosen item is sampled from the softmax:

$$u_i(S) = \boldsymbol{\theta} \cdot \mathbf{x}_i + s\,\lambda\, c_i(S). \tag{2.5}$$

For LCL data (D0), $c_i(S) = \mathbf{x}_i^\top A \bar{\mathbf{x}}_S$ with $A = UV^\top$ of rank 2 ($U, V$ random $6 \times 2$) scaled to Frobenius norm 1. For the higher-order datasets only the first two features are used as a position $\mathbf{p}_i$ on a plane, and for an effect of order $k$ we add up how close item $i$ is to the centre of every group $T$ of $k$ other items:

$$c_i(S) = \sum_{T \subseteq S\setminus\{i\},\, |T|=k} K(\|\mathbf{p}_i - \bar{\mathbf{p}}_T\|^2), \qquad \bar{\mathbf{p}}_T = \tfrac{1}{k}\textstyle\sum_{j\in T}\mathbf{p}_j, \tag{2.6}$$

with a Gaussian closeness $K(d^2) = e^{-d^2/\sigma^2}$, $\sigma = 0.2$, or, in the box versions, $K = 1$ if $d < 0.25$ and 0 otherwise. Each term depends on item $i$ and $k$ others jointly, so the effect is of order $k$ (Table 2.2). For example, with $k = 2$ an item gains when it lies close to the midpoint of some pair of other items, which is how we plant a compromise effect; with $k = 1$ and a negative sign, an item loses for every look-alike, which is a similarity effect.

**Table 2.2.** Datasets. The box versions D1b–D3b have the same order and sign as D1–D3 and their own $\lambda$, set by the same rule. D4 and D5 are used only in Section 3.5.

| Dataset | Order $k$ | Sign $s$ | $\lambda$ | What it means |
|---|---|---|---|---|
| D0 (LCL) | 1, linear | +1 | 1.3120 | the slate mean changes how features are valued |
| D1 (similarity) | 1 | −1 | 1.8310 | an item loses utility for every look-alike |
| D2 (compromise) | 2 | +1 | 0.5087 | an item gains when it lies between two others |
| D3 (centre) | 3 | +1 | 0.2507 | an item gains at the centre of three others |
| D4, D5 | 4, 5 | +1 | 0.1440, 0.1098 | the same for groups of four and five |

**Strength.** If an effect is too strong every model learns it, and if too weak none can find it. Part of every higher-order effect can be imitated by simple slate statistics, because the centres of random groups crowd towards the slate mean (the *concentration effect*). In our calibration, about 30% of the effect could be explained by the slate's mean and standard deviation at order 2, about 61% at order 3 and 72–73% at orders 4 and 5; in the final runs, the trained mean+sd pool closes 31% of the gap on D2 and 70% on D3. We therefore set $\lambda$ so that the *hard part*, the part the mean+sd pool cannot explain, is about 0.03 nats for every higher-order dataset (in the final runs the pool ends 0.023 to 0.032 nats above the oracle); for D0 the whole context effect is about 0.05 nats. Each dataset has 285,000 training, 15,000 validation and 50,000 test slates.

## 2.5 Metrics and a Fair Comparison

**Metrics.** Our basic measure is the test cross-entropy (CE) against the true choice probabilities: it tells how far the model's probabilities are from the true ones, so lower is better, and no model

can go below the oracle. A raw CE value is hard to read, so we compare each model with the oracle, the twin and the mean+sd pool, which gives two percentages:

$$\text{gap to twin closed} = \frac{\text{CE}_{\text{twin}} - \text{CE}_{\text{model}}}{\text{CE}_{\text{twin}} - \text{CE}_{\text{oracle}}}, \qquad \text{hard part captured} = \frac{\text{CE}_{\text{pool}} - \text{CE}_{\text{model}}}{\text{CE}_{\text{pool}} - \text{CE}_{\text{oracle}}}. \tag{2.7}$$

The first is the share of the whole context effect a model recovers; the second the share of the hard part. 100% is the oracle, 0% is no better than the control and a negative value is worse. Since $\log P(i \mid S) - \log P(j \mid S) = u_i - u_j$, a model can beat the twin only by getting the context-dependent part of these differences right; Section 3.4 also tests this pair by pair.

**Fair comparison.** Both models see the same data and are trained the same way (Adam, learning rate $10^{-3}$ then $10^{-4}$, batch 512, early stopping on validation loss). Both families were tuned over 22 configurations in total on 7 datasets, choosing on seeds 100–104 and reporting on a separate set of 30 seeds (0–29), to avoid picking a setting that only looked good by chance. We also ran size-matched controls and compare models seed by seed, reporting how many of the 30 seeds each one wins.

# 3 Experiments and Results

## 3.1 First-Order Context: Recovering the LCL Matrix

Our earlier study (the thesis in our repository) asked RQ1 on LCL data with $d = 16$ features, $A$ of rank 5, five items per slate and 285,000 training sessions, over 30 datasets. A *behavioural probe* reads $\hat{A}$ and $\hat{\boldsymbol{\theta}}$ from any trained model without looking at its weights: it keeps two items fixed, refills the rest of the slate many times, and fits the LCL form (Equation 2.1) to the change in their score gap. On the generator's own scores the probe is exact to $10^{-15}$, and on a context-free model it returns $\hat{A} = 0$.

**Table 3.1.** Prediction and recovery on LCL data (30 datasets, mean; ± s.d. for the error). Excess NLL and KL in units of $10^{-3}$. Error = $\|\hat{A} - A\|/\|A\|$; size = $\|\hat{A}\|/\|A\|$.

| | Prediction | | Recovery of $A$ | | | $\boldsymbol{\theta}$ |
|---|---|---|---|---|---|---|
| Model | Excess NLL | KL | Error | Cosine | Size | Rel. error |
| Oracle LCL | 0.00 | 0.69 | 0.092 | 0.996 | 1.01 | 0.020 |
| Self-attention, 1 layer | 4.78 | 5.37 | 0.150 ± 0.040 | 0.990 | 0.93 | 0.042 |
| Self-attention, 6 layers | 4.50 | 5.38 | 0.163 ± 0.062 | 0.987 | 0.93 | 0.046 |
| Mean-pool, 2 layers | 4.14 | 4.72 | 0.142 ± 0.016 | 0.991 | 0.95 | 0.032 |
| Flat MLP, 2 layers | 22.06 | 22.51 | 0.241 ± 0.035 | 0.979 | 0.85 | 0.026 |

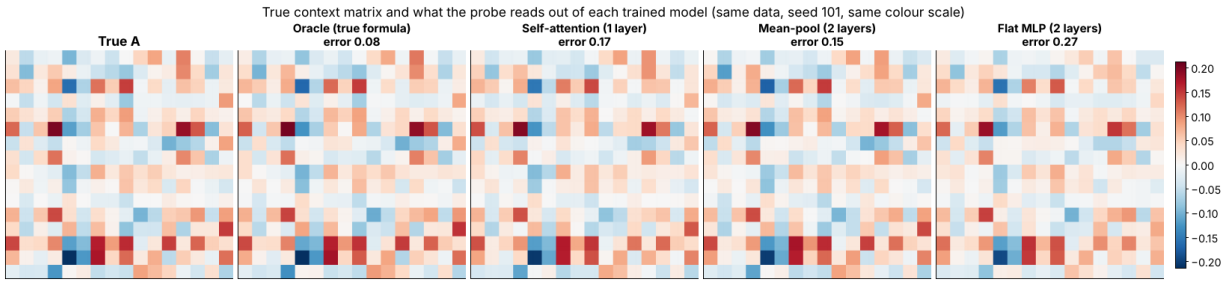



**Figure 3.1.** The true $A$ of one LCL dataset and the $\hat{A}$ read from each trained model, on the same colour scale.

Self-attention that predicts well has also learned the context rule (Table 3.1, Figure 3.1): its $\hat{A}$ has the right pattern (cosine 0.990), about 93% of the true strength, and its error is 1.5 to 1.6 times the

floor of the oracle. It is within noise of the mean-pool network and clearly better than a flat MLP of the same size. Trained on data with no context effect, it reports $\|\hat{A}\| = 0.033$ against 0.93 with the effect, so it does not invent context. Recovery depends on the amount of data (Figure 3.2), the rank of $A$ and the number of features, and not on depth, sparsity, learning rate or weight decay; about 40% of the remaining squared error of self-attention comes from the random initialisation, and averaging 20 trained models brings its error from 0.144 to 0.110. Among models that fit the usual data equally well, those that recovered $A$ better also predicted unusual slates better (partial correlation 0.73).

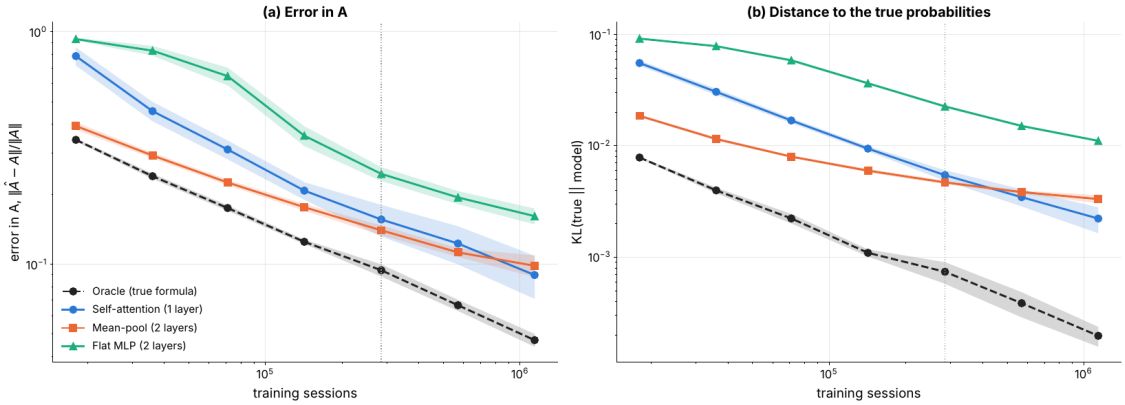



**Figure 3.2.** LCL data: error in $\hat{A}$ (a) and KL to the true probabilities (b) against the number of training sessions (log scales, 15 datasets per point; dotted line: reference size). Self-attention starts far behind but keeps improving and catches up with mean-pool at the largest sizes.

## 3.2 Higher-Order Context: Self-Attention against DeepHalo

Now let us come to the main question of this report: what happens when the context effect involves two or three other items together?

**First order.** On D0, all models tie: self-attention, DHQ and the mean+sd pool each close about 97% of the gap to the twin. An LCL effect depends only on the slate mean, which every one of these models can compute.

**Orders 2 and 3.** On D2 and D3 the tuned self-attention model has a lower test loss than the tuned DHQ model on *all* 30 seeds, and captures 21.6% and 32.3% of the hard part against 9.8% and 7.3% (Figure 3.3). Even the better model is far from the oracle.

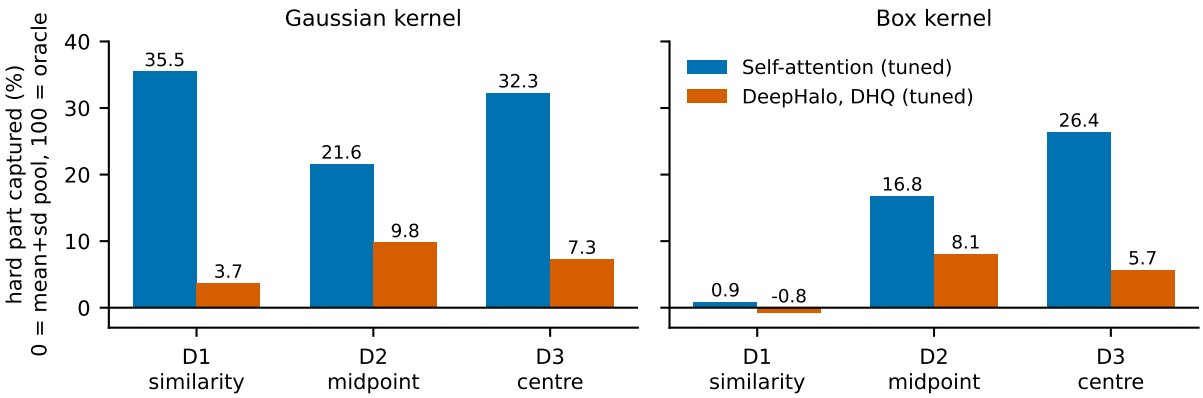



**Figure 3.3.** Share of the hard part captured by the tuned models, mean over 30 seeds. Left: Gaussian kernel. Right: box kernel, where attention has no built-in shortcut.

**Robustness: does the kernel matter?** The Gaussian kernel can be written in a form that

looks like the $\exp(\mathbf{q} \cdot \mathbf{k})$ inside softmax attention, which could give attention a shortcut. With the box kernel, attention's large lead on similarity (35.5% against 3.7% on D1) *disappears* (0.9% against −0.8% on D1b): neither tuned model learns box-kernel similarity in any useful amount, so the D1 result most likely came from the Gaussian kernel and not from a real ability to learn similarity. At orders 2 and 3 the advantage *survives* (16.8% against 8.1% on D2b, 26.4% against 5.7% on D3b), again on all 30 seeds.

**Depth or width?** A 2-layer attention model beat a 1-layer model made wider to the same size (width 46) on D2 and D3 on all 30 seeds, so the gain comes from depth, not from extra parameters. Tuning over 22 configurations moved the losses only a little, which pointed to a limit of data rather than of capacity.

## 3.3 Training-Set Size

Since tuning did not close the gap to the oracle, the natural next question is whether more data does. We trained the tuned models, the pool and the twin on 11,000 to 1,000,000 training slates, with the same validation and test sets at every size and nested training sets (30 seeds per size, 10 at one million). The gap to the oracle closes steadily with data and shows no sign of levelling off (Figure 3.4).

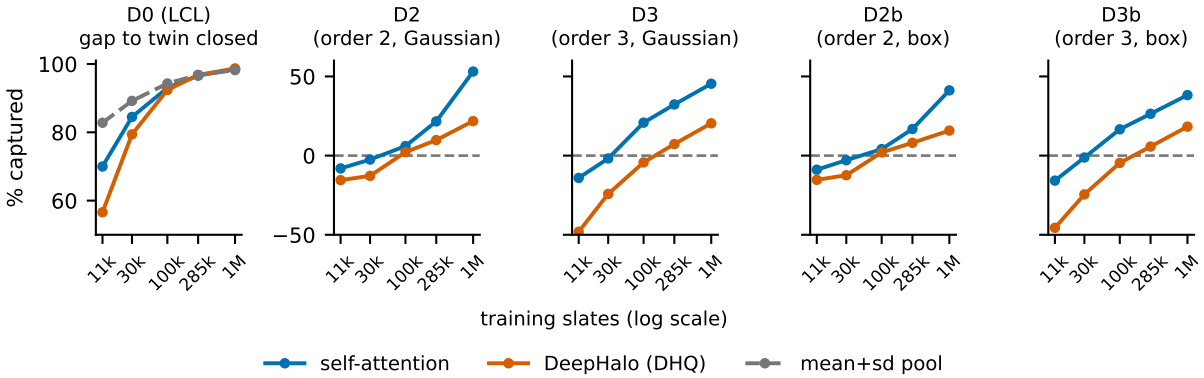



**Figure 3.4.** Share captured against training-set size (mean over 30 seeds, 10 at 1M). D0: share of the whole context effect; others: share of the hard part, where the dashed line at 0 is the mean+sd pool.

1. **More data keeps helping.** On D2 self-attention captures −8.1% of the hard part at 11,000 slates, 21.6% at 285,000 and 53.1% at 1,000,000; on D3 it goes from 32.3% at 285,000 to 45.4% at 1,000,000. Tuning the size and depth of the models moved the results only a little, while more data moved them a lot, so the remaining gap is mainly a data limit.
2. **Below about 100,000 slates higher-order context is not learnable.** At 11,000 and 30,000 slates both models are, on average, *worse* than the pool on every higher-order dataset (negative values in Figure 3.4); self-attention first beats the pool on every one of the 30 seeds at 100,000 slates.
3. **Self-attention needs less data than DeepHalo, and its lead grows.** At 100,000 slates DHQ is still below the pool on D3 and D3b, while self-attention captures 20.8% and 16.6%; at one million it captures 53.1% against 21.8% on D2 and 45.4% against 20.4% on D3.
4. **First-order data favour the simplest model when data are scarce.** On D0 the pool, whose averaging matches LCL exactly, closes 82.8% of the gap at 11,000 slates against 70.0% for self-attention and 56.6% for DHQ; all three reach about 98.5% at one million.

## 3.4 A Look Inside the Models

So far we have only looked at the loss. In this section we open the trained models and ask *what* they have learned and *where*. For this we retrained four models on D0, D2, D3 and the box versions

D2b, D3b with 10 seeds, saving their weights: the tuned 3-layer self-attention model, a 1-layer one (width 32, 2 heads), the tuned DHQ and the pool. The retrained models reproduced the final test losses exactly.

**Recovering $\boldsymbol{\theta}$ and $A$ on LCL data.** Fitting the LCL form to each model's own utilities, the fitted $A$ matches the true $A$ with correlation 0.999 for every model and the fitted $\boldsymbol{\theta}$ has cosine 1.000; the LCL form explains 99.4–99.6% of each model's utilities. The size of $\hat{A}$ is 0.986 of the truth for DHQ, 0.964 and 0.961 for 3- and 1-layer attention and 0.966 for the pool, so attention under-estimates the effect by about 4%, consistent with Section 3.1.

**Where context is computed.** We replaced the attention of one layer at a time by a plain average over all items, and measured the rise in test loss as a share of the model's context gain over its twin (Table 3.2). On LCL data averaging costs almost nothing, and the heads are almost uniform (spread 0.98–0.99, where 1 is uniform; weight on itself about 0.10, i.e. 1/10): for LCL, attention simply computes the slate mean. At orders 2 and 3 averaging costs a lot, so selective attention is needed, and in the 3-layer model it sits mainly in layers 2 and 3. Letting each item attend only to itself instead raises the loss by *more* than the whole context gain (for example 462% for the 1-layer model on D0): the trained network depends on its context input, so this measures dependence rather than the value of context.

**Table 3.2.** Loss rise when attention is replaced by a plain average, as % of the context gain (10 seeds).

| Replaced in | D0 (LCL) | D2 | D3 | D2b (box) | D3b (box) |
|---|---|---|---|---|---|
| 3-layer model, layer 1 | 1.7 | 12.0 | 15.6 | 14.5 | 15.3 |
| 3-layer model, layer 2 | 0.5 | 140.8 | 25.0 | 146.3 | 23.6 |
| 3-layer model, layer 3 | 0.8 | 102.7 | 41.1 | 141.6 | 47.0 |
| 3-layer model, all layers | 4.1 | 248.8 | 83.2 | 274.3 | 88.2 |
| 1-layer model | 3.4 | 138.1 | 80.5 | 131.6 | 82.1 |

**What the heads attend to.** The correlation between the attention weight from item $i$ to item $j$ and how close they are in the two context features is 0.61 and 0.53 (two heads of the 1-layer model) on D2 and 0.59 and 0.60 on the box version D2b; on D3 and D3b it is 0.44/0.37 and 0.42/0.39. Closeness in the other four features gives correlations between $-0.03$ and 0.00. So the heads learn to look at the items that are close in exactly the features that define the effect, also when the kernel shortcut is removed; in the 3-layer model the strongest focusing is in layer 2.

**A direct IIA test.** For each test slate we kept items 0 and 1, replaced the other eight with new random items, and compared the change in $u_0 - u_1$ under the truth and under the model (Table 3.3); under IIA this change would be zero.

**Table 3.3.** IIA test: correlation / slope between the true and the model's change in $u_0 - u_1$ (10 seeds; slope 1 = right size).

| Model | D0 (LCL) | D2 | D3 | D2b (box) | D3b (box) |
|---|---|---|---|---|---|
| Self-attention, 3 layers | 0.981 / 0.959 | 0.687 / 0.555 | 0.890 / 0.841 | 0.624 / 0.477 | 0.864 / 0.789 |
| Self-attention, 1 layer | 0.985 / 0.957 | 0.636 / 0.452 | 0.860 / 0.766 | 0.573 / 0.368 | 0.835 / 0.735 |
| DeepHalo (DHQ) | 0.988 / 0.983 | 0.616 / 0.461 | 0.833 / 0.774 | 0.563 / 0.395 | 0.806 / 0.752 |
| Mean+sd pool | 0.984 / 0.964 | 0.554 / 0.348 | 0.827 / 0.714 | 0.504 / 0.297 | 0.801 / 0.679 |

On LCL data all models pass almost perfectly. At orders 2 and 3 the 3-layer attention model is best, the 1-layer model and DHQ come next and the pool is last, the same order as the hard-part results; all slopes are below 1, so every model under-reacts to a change of context.

## 3.5 Depth and the Order of Interaction

Does a deeper attention network learn higher orders? A simple counting argument of our own (not yet checked against the literature) says that in a network made only of linear attention layers, the

number of other items that can act together on one utility grows from 1 after one layer to at most $2^L - 1$ after $L$ layers. A nonlinear feed-forward layer between attention steps can raise the order by itself (squaring a sum over items gives terms in pairs of items), so the clean counting holds only for attention-only models. On D2–D5 we trained, at depths 1 to 4 with the parameters fixed within each family, the full self-attention model (about 26k parameters) and attention-only models with a per-item MLP embedding followed only by attention layers with residual connections, with softmax or linear attention (about 16k parameters); 5 seeds, 285,000 slates.

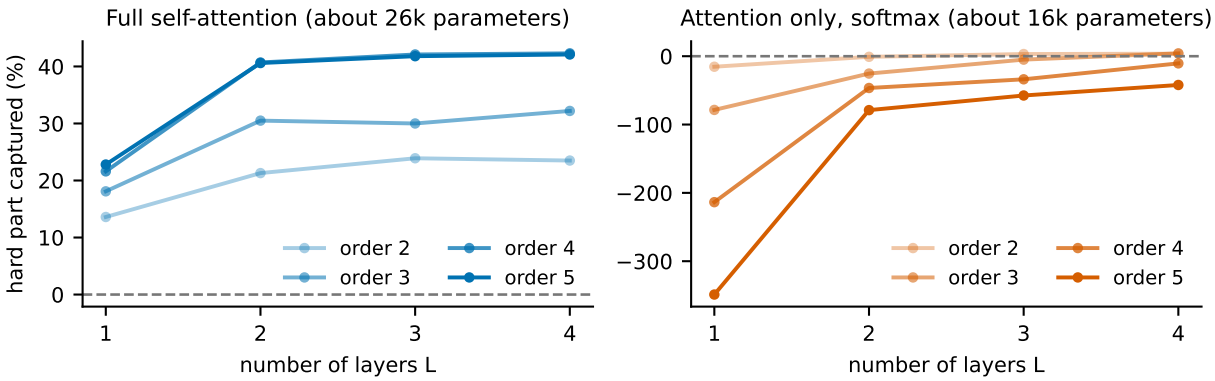



**Figure 3.5.** Hard part captured against depth for effects of order 2 to 5 (mean over 5 seeds). Darker lines are higher orders.

For full self-attention, going from 1 to 2 layers gives a large gain at every order and 3 or 4 layers add little (Figure 3.5); even one layer captures part of an order-5 effect, as expected since the feed-forward network raises the order. The gain from the second layer grows with the order (7.7 points at order 2, 12.4, 19.1 and 17.8 at orders 3 to 5), but we see no doubling pattern. The attention-only models are worse than the pool almost everywhere: softmax attention improves with depth but only just becomes positive at depth 3–4 on order 2 (3.2%, 3.7%) and at depth 4 on order 3 (4.2%), and linear attention never clearly beats the pool (for example −0.4% at depth 2 on order 2, where by the counting it has enough capacity). So the test of $2^L - 1$ is inconclusive: we cannot yet tell whether these models fail because of what they can represent or because they are hard to train without LayerNorm and with fewer parameters.

## 3.6 Real Data: LPMC and Expedia

We applied the same model families and controls to two real datasets used in the DeepHalo paper (Table 3.4). On LPMC the split holds out whole households and a survey year; on Expedia every model has a separate learned position term, so a context gain cannot come from position, and only randomised rankings are used.

**Table 3.4.** Real datasets.

| | LPMC | Expedia |
|---|---|---|
| Choice | travel mode of a trip (walk, cycle, public transport, drive) | hotel booked in a search |
| Items per set | 4, always all available | 5 to 38 (mean 23.8) |
| Features | 8 per mode + 17 traveller features | 14 per hotel + 10 search features |
| Train / val / test | 49,327 / 5,439 / 26,320 trips (test = year 3) | 11,092 / 1,584 / 3,169 searches (by time) |

**LPMC: no detectable context.** Over 15 paired seeds, self-attention minus its twin is $-0.0007 \pm 0.0012$, better on only 9 of 15 seeds. All four modes are always available, so the choice set never changes and context can act only through the other modes' attributes. Our DeepHalo reproduces

**Table 3.5.** Test NLL on real data, with parameters counted as used by the model (mean over seeds: 15 for self-attention, its twin, the pools and tuned DHQ; 5 for the other neural models; 1 for MNL and LCL).

| LPMC (held-out year) | | | Expedia (latest searches; uniform guess 3.0282) | | |
|---|---|---|---|---|---|
| Model | Params | NLL | Model | Params | NLL |
| MNL | – | 0.8843 | MNL | 55 | 2.6343 |
| DeepHalo, released code | 42,369 | 0.6881 | DeepHalo, paper size | 52,210 | 2.6069 |
| Context-free MLP | 9,606 | 0.6822 | LCL | – | 2.5965 |
| DeepHalo, size-matched | 10,188 | 0.6805 | Self-attention twin | 9,482 | 2.5761 |
| Self-attention twin | 9,473 | 0.6797 | Pool: mean + sd | 9,588 | 2.5693 |
| Self-attention, 1 layer | 9,473 | 0.6790 | Pool: z-score | 9,358 | 2.5617 |
| | | | DeepHalo, tuned (DHQ) | 2,162 | 2.5585 |
| | | | Self-attention, 1 layer | 9,482 | 2.5584 |

the paper on its random split (0.6404 against 0.6407 reported), but our context-free MLP reaches 0.6382 there against 0.6870 reported for the paper's MLP, so DeepHalo's reported advantage over a context-free model is not reproduced; our MLP sees the traveller features, and the paper's MLP inputs should be checked.

**Expedia: a reliable but shallow context gain.** Self-attention beats its twin by $0.0177 \pm 0.0017$ nats on all 15 seeds, and DeepHalo gains $0.0092 \pm 0.0023$ over its own blindfolded version. Tuned against tuned the two tie ($-0.0032 \pm 0.0023$), and DeepHalo at the paper's size overfits. A pool that sees each hotel's features standardised against the others in the same search (z-score) recovers 0.0144 of attention's 0.0177-nat gain, about 81%; the rest ($-0.0034 \pm 0.0020$) is not significant. So nothing beyond low-order slate statistics is reliably detected. This is not proof that real choices have no higher-order effects: Expedia's 11,092 training searches correspond to the 11,000 row of Section 3.3, where even a planted higher-order effect could not be learned.

### 3.7 Lessons Learned Along the Way

Some early conclusions did not survive careful checks, and each was corrected by adding a control: our first claim that attention beats DeepHalo everywhere partly came from using its weaker linear form; the similarity advantage on D1 most likely came from the Gaussian kernel; and an early claim that DeepHalo never goes beyond the mean+sd summary was false for the quadratic version. All numbers in this report are from the corrected runs.

## 4 Conclusion, Limitations and Next Steps

In one line: self-attention does learn context effects from choices alone, it learns higher-order effects better than DeepHalo, and what holds it back is mainly the amount of data.

**RQ1.** On first-order LCL data, self-attention that predicts well has also learned the context rule: cosine 0.990 to the true $A$, about 93% of its strength and no false context, as good as a mean-pool network. For LCL it simply averages the slate.

**RQ2.** At orders 2 and 3, tuned self-attention beats tuned DeepHalo on every seed and with two kernels (16.8–32.3% of the hard part against 5.7–9.8%); its heads attend to the items close in the features that define the effect, and a direct IIA test ranks the models the same way. Similarity effects with a box kernel are learned by neither tuned model.

**RQ3.** Data matters most: going from 285,000 to 1,000,000 slates raises the hard part captured from 21.6% to 53.1% at order 2 and from 32.3% to 45.4% at order 3, and below about 100,000 slates no model learns higher-order context. Depth beats width, and the second layer helps more for higher orders, but more than two layers adds little; attention without its feed-forward network learns very little context.

**RQ4.** On LPMC no context is detected; on Expedia a small context gain exists but is mostly explained by comparing each hotel with the others in the search. To test higher-order effects on real data we need much larger datasets that actually contain them.

**Limitations.** The main results are on synthetic data with 10 items and 6 features, with each order planted on its own, while real data will mix orders. Our metric is based on average loss; the IIA test and the look inside the models support it but use 10 seeds rather than 30. The attention-only models of Section 3.5 learned too little to test the depth–order idea.

**Next steps.**

1. Extend the training-size sweep beyond 1,000,000 slates and to mixed-order data, with partial oracles that know only some orders.
2. Run the analyses of Section 3.4 on all 30 seeds and add DeepHalo's own order-wise readout to compare the two models order by order.
3. On Expedia, compute order-wise relative context effects of the trained models, and use the size sweep to say how large an effect must be to be detectable with about 11,000 searches.
4. Explain *how* attention stores the effect, for example whether its value and output weights with the readout reproduce $A$ on LCL data, and why reading attention weights directly differs from behavioural probing.

# Bibliography


Batsell, R. R. and Polking, J. C. (1985). A new class of market share models. *Marketing Science*, 4(3):177–198.

Bradley, R. A. and Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345.

Evers, E. R. K. and Lakens, D. (2014). Revisiting Tversky's diagnosticity principle. *Frontiers in Psychology*, 5:875.

Goldstone, R. L., Medin, D. L., and Halberstadt, J. (1997). Similarity in context. *Memory & Cognition*, 25(2):237–255.

Huber, J., Payne, J. W., and Puto, C. (1982). Adding asymmetrically dominated alternatives: Violations of regularity and the similarity hypothesis. *Journal of Consumer Research*, 9(1):90–98.

Kajitsuka, T. and Sato, I. (2024). Are transformers with one layer self-attention using low-rank weight matrices universal approximators? In *ICLR 2024*.

Ko, J. and Li, A. A. (2023). Modeling choice via self-attention. arXiv:2311.07607.

Luce, R. D. (1959). *Individual Choice Behavior: A Theoretical Analysis*. Wiley.

McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior. In *Frontiers in Econometrics*, pages 105–142. Academic Press.

Pfannschmidt, K., Gupta, P., Haddenhorst, B., and Hüllermeier, E. (2022). Learning context-dependent choice functions. *International Journal of Approximate Reasoning*.

Pobrotyn, P., Bartczak, T., Synowiec, M., Białobrzeski, R., and Bojar, J. (2020). Context-aware learning to rank with self-attention. In *SIGIR eCom Workshop*.

Seshadri, A., Peysakhovich, A., and Ugander, J. (2019). Discovering context effects from raw choice data. In *ICML 2019*, PMLR 97.

Simonson, I. (1989). Choice based on reasons: The case of attraction and compromise effects. *Journal of Consumer Research*, 16(2):158–174.

Tomlinson, K. and Benson, A. R. (2021). Learning interpretable feature context effects in discrete choice. In *KDD 2021*.

Tversky, A. (1972). Elimination by aspects: A theory of choice. *Psychological Review*, 79(4):281–299.

Tversky, A. (1977). Features of similarity. *Psychological Review*, 84(4):327–352.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *NeurIPS 30*.

Wang, H., Li, X., and Talluri, K. (2023). Transformer choice net: A transformer neural network for choice prediction. arXiv:2310.08716.

Zhang, S., Wang, Z., Gao, R., and Li, S. (2025). DeepHalo: A neural choice model with controllable context effects. In *NeurIPS 2025*. arXiv:2601.04616.